Software Requirements and Design Document

For

Group <27>

Version 1.0

Authors:
Carter V
Nathanael T
Aryan P
Julian S
Jack N

1. Overview (5 points)

The system is a 2D action-adventure metroidvania game built in C# using the Unity engine. It is designed to emulate the exploration and key/lock style of gameplay found in other metroidvanias like Super Metroid and Hollow Knight. It will emphasize responsive controls and smooth character animations. It will obviously be designed with object oriented principles.

2. Functional Requirements (10 points)

1. It should have responsive character movement (left right jump), including ledge grabbing. - high
2. It should have omnidirectional aiming and shooting mechanics - high
3. It should have player animations based on current movement state and weapon aiming position - high
4. It should have readable enemy ai (with sprites) - high
5. It should have multiple area types with multiple levels - high
6. It should have a player HUD - high
7. It should have sound effects for weapons movement, and some environment - high
8. It should have a main menu - medium
9. It should have a save/load system - relatively low

3. Non-functional Requirements (10 points)

1. Game should support new additions without refactoring
2. Game should run at least 60 fps
3. Game should not crash during gameplay
4. HUD elements should be at least 1080p
5. Code should utilize OOP principles for maximum reusability
6. Sound should be synchronized to actions
7. Saves should be stored in a format that prevents accidental corruption.

4. Use Case Diagram (10 points)

Actor: Player

Use Cases:

Move:
<<include>> Jump:
<<extend>> Double Jump

Attack:
<<include>> Detect Hit

Interact with Environment:
<<extend>> Open Door

Collect Items:
<<extend>> Unlock Ability

Save Game:

Load Game:

5. Class Diagram and/or Sequence Diagrams (15 points)

CLASS DIAGRAMS

This will be a rough overview, considering the complexity of the project

Entity:

Attributes:

1. Health, speed

Sub types:

Player:

a. Attributes:
    i. oxygen, ammo
b. Methods: movement and collision, health and abilities, UI interaction, Spawn/death, melee and ranged attacks (ranged interacts with projectile class)
c. Related: Player state handler (allows player to switch behavior states), player animator

Enemy

a. Attributes:
    i. Patrol points, vision length
b. Methods:
    i. Patrol and attack

Levels:

Attributes:
1. Enemy types, items, connected levels

Methods:
1. Load level, spawn enemies (interacts with Entity type classes

Items:

Attributes:
1. Type

Methods:
1. PickUp() This will interact with the player scripts, probably just enabling bools

HUD/UI:

Not entirely decided, but player related HUD must interact with the player scripts, likely just containing a reference to the player to access health and ammo data. Will also interact with save/load system.

Game Handler:

Attributes:
1. Bool isPaused (not sure whether this is something unity could handle itself)

Methods:
1. saveGame(), loadGame(), pause(). Interacts with UI in particular

SEQUENCE DIAGRAMS:

1. Left and right movement:
Player->PlayerStateMachine
PlayerStateMachine - > PlayerMovement.Update()
Update()->horizontalMovement()
horizontalMovement()->collision()

2. Shooting
Player->input
Shooting->create projectile
Shooting -> initialize speed and direction

Projectile->move - > detect collision

Projectile - > handle collisions (destroy)

3. Jumping
Player->Input->check if grounded->apply jump force

6. Operating Environment (5 points)

A PC environment, specifically windows (at least 10 and 11), however we are currently unsure of specific versions or other software dependencies beyond specified. Unity might provide the tools to include other platforms like mac, linux, mobile, and consoles, but there is currently no plans to compensate for these platforms.

7. Assumptions and Dependencies (5 points)

- Possible usage of out-sourcing music, if time constraints are too much.

No other Assumptions or Dependencies are currently identified.